

Roll no: **43221**

Title: **Assignment 4: Use Autoencoder to implement anomaly detection. Build the model by using:**

- Import required libraries
- Upload / access the dataset
- Encoder converts it into latent representation
- Decoder networks convert it back to the original input
- Compile the models with Optimizer, Loss, and Evaluation Metrics

Dataset: CreditCard

a. Import required libraries

```
In [ ]: import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
```

```
2023-11-04 04:20:16.280013: I
tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
```

b. Upload / access the dataset

```
In [ ]: dataset = pd.read_csv("Datasets/creditcardfraud-csv/creditcard.csv")
```

```
In [ ]: # Preprocess the data (if needed)
# You may need to normalize the data, handle missing values, etc.
# For simplicity, we'll perform normalization in this example
# Normalize the data to have a mean of 0 and a standard deviation of 1
scaler = StandardScaler()
X = scaler.fit_transform(dataset.drop("Class", axis=1))
y = dataset["Class"]
```

```
In [ ]: # Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
In [ ]: # Build and train the Autoencoder model
input_dim = X_train.shape[1]
```

c. Encoder converts it into latent representation

```
In [ ]: # Encoder
encoder = models.Sequential([
    layers.Input(shape=(input_dim,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(16, activation='relu')
])
```

d. Decoder networks convert it back to the original input

```
In [ ]: # Decoder
decoder = models.Sequential([
    layers.Input(shape=(16,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(input_dim, activation='linear') # Using 'linear'
activation for reconstruction
])

# Autoencoder
autoencoder = models.Sequential([
    encoder,
    decoder
])
```

e. Compile the models with Optimizer, Loss, and Evaluation Metrics

```
In [ ]: autoencoder.compile(optimizer='adam', loss='mean_squared_error')
autoencoder.fit(X_train, X_train, epochs=10, batch_size=32, shuffle=True,
validation_data=(X_test, X_test))
```

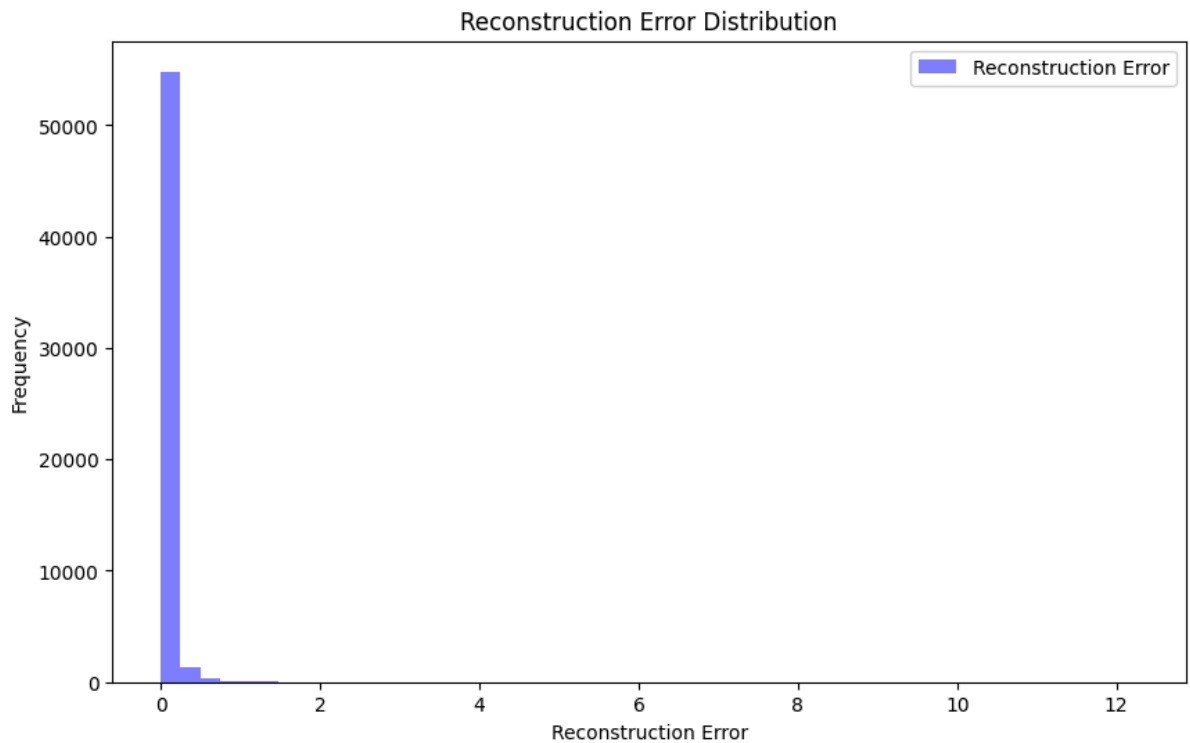
```
Epoch 1/10
7121/7121 [=====] - 11s 1ms/step - loss: 0.3106 -
val_loss: 0.1632
Epoch 2/10
7121/7121 [=====] - 10s 1ms/step - loss: 0.1349 -
val_loss: 0.1027
Epoch 3/10
7121/7121 [=====] - 10s 1ms/step - loss: 0.0918 -
val_loss: 0.0888
Epoch 4/10
7121/7121 [=====] - 11s 1ms/step - loss: 0.0788 -
val_loss: 0.0702
Epoch 5/10
7121/7121 [=====] - 10s 1ms/step - loss: 0.0712 -
val_loss: 0.0692
Epoch 6/10
7121/7121 [=====] - 11s 1ms/step - loss: 0.0661 -
val_loss: 0.0604
Epoch 7/10
7121/7121 [=====] - 10s 1ms/step - loss: 0.0630 -
val_loss: 0.0794
Epoch 8/10
7121/7121 [=====] - 11s 1ms/step - loss: 0.0604 -
val_loss: 0.0623
Epoch 9/10
7121/7121 [=====] - 10s 1ms/step - loss: 0.0579 -
val_loss: 0.0530
Epoch 10/10
7121/7121 [=====] - 10s 1ms/step - loss: 0.0575 -
val_loss: 0.0558
```

```
Out[11]: <keras.src.callbacks.History at 0x7f2396c5bf10>
```

```
In [ ]: # Detect anomalies and tune the threshold
y_pred = autoencoder.predict(X_test)
mse = np.mean(np.power(X_test - y_pred, 2), axis=1)
```

```
1781/1781 [=====] - 2s 930us/step
```

```
In [ ]: # Visualize the reconstruction error distribution
plt.figure(figsize=(10, 6))
plt.hist(mse, bins=50, alpha=0.5, color='b', label='Reconstruction Error')
plt.xlabel("Reconstruction Error")
plt.ylabel("Frequency")
plt.legend()
plt.title("Reconstruction Error Distribution")
plt.show()
```



```
In [ ]: # Threshold tuning (iterate and adjust as needed)
thresholds = np.arange(0.1, 1.0, 0.1) # Adjust the step size as needed

for threshold in thresholds:
    anomalies = mse > threshold
```

```
In [ ]: # Count the number of anomalies
num_anomalies = np.sum(anomalies)
print(f"Threshold: {threshold:.1f}, Number of anomalies: {num_anomalies}")
```

Threshold: 0.9, Number of anomalies: 291

```
In [ ]: # Evaluate the model
print("Confusion Matrix:")
print(confusion_matrix(y_test, anomalies))

print("\nClassification Report:")
print(classification_report(y_test, anomalies))
```

Confusion Matrix:

```
[[56610  254]
 [   61   37]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.13	0.38	0.19	98
accuracy			0.99	56962
macro avg	0.56	0.69	0.59	56962
weighted avg	1.00	0.99	1.00	56962

```
In [ ]: import seaborn as sns
```

```
In [ ]: plt.figure(figsize = (6, 4.75))
sns.heatmap(confusion_matrix(y_test, anomalies), annot = True, annot_kws =
{"size": 16}, fmt = 'd')
plt.xticks([0.5, 1.5], rotation = 'horizontal')
plt.yticks([0.5, 1.5], rotation = 'horizontal')
plt.xlabel("Predicted label", fontsize = 14)
plt.ylabel("True label", fontsize = 14)
plt.title("Confusion Matrix", fontsize = 14)
plt.grid(False)
plt.show()
```

